Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO2 – Debugging and Optimisation Task
Course Outcome Assessed	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems.
Maximum Marks	**50**
Duration	1.5 hrs (Assessment conducted in class)
Mode	Individual
Student Reg. No.	192511230
Student Name	ALURI POOJITHA

Instructions: Debug the code by reasoning and testing, not by rewriting it from scratch. For every change, state what was wrong, why it was wrong and how you verified the fix. Optimisations must preserve the corrected outputs. Marks for each part are shown in brackets.

DEBUGGING AND OPTIMISATION TASK (50 MARKS)

Scenario – SOC Alert-Triage Module of "SentinelOps": SentinelOps is the Security Operations Centre (SOC) of a cloud-services company. A junior developer wrote the Python module below to (1) sort incoming alerts by severity, highest first, keeping alerts of equal severity in arrival order; (2) check whether a source IP address is on the blocklist; (3) find every position of an attack signature in a log line; and (4) find the two login attempts whose locations are closest to each other, which the SOC uses to flag clustered credential-stuffing attacks. The module passed code review but failed in production: alerts disappeared or were duplicated, blocked IP addresses got through, signatures at the end of a log were missed while false signatures were reported, and the "closest logins" report flagged logins hundreds of kilometres apart. The module must be fixed before it processes about 50,000 alerts and batches of 8,000 login events per day.

Code under investigation:

```
import math

def sort_alerts(alerts):            # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j > 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j] = key
    return alerts

def is_blocked(blocklist, ip):      # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return False
        else:
            return False

def find_signature(log, pattern):   # brute-force string matching, all occurrences
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        if j == m - 1:
            hits.append(i)
    return hits

def closest_logins(events):         # brute-force closest pair of login locations
    min_dist = 0
    pair = None
    for i in range(len(events)):
        for j in range(len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d > min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist
```

Test data:

```
alerts    = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries   = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log, pattern = "FAILFAIXFAIL", "FAIL"
events    = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]   # (x, y) in
km
```

(a) Reproduce and Identify the Defects (10 Marks): Run the module on the test data. For each function, record the expected output and the actual output, then locate every defect: state the function, the faulty line and the nature of the error. The module contains nine defects, and none of them is marked in the code.

(b) Explain the Root Causes and Debug Systematically (12 Marks): For each defect, explain its root cause and how it produces the observed wrong output. Correct the functions one at a time, supporting your reasoning with a step-by-step trace (for example, the insertion trace for the alerts list and the alignment table for the log), and re-test after every correction. Include boundary tests such as an empty blocklist, a pattern at the very end of the log, a pattern longer than the log, equal severities and fewer than two login events.

(c) Optimise the Corrected Module (10 Marks): Without changing any output, improve the corrected module for the production workload. Justify whether insertion sort remains suitable for nearly sorted alerts; propose a better approach for repeated blocklist lookups; make sure the signature scan performs no redundant comparisons; and, for closest_logins, eliminate self and

repeated pairs, remove unnecessary square-root computations and prune pairs that cannot be closest.

(d) Analyse Before and After (10 Marks): Derive the number of basic operations and the time complexity of each function before and after optimisation. Measure the distance computations of the brute-force and optimised closest_logins for at least three batch sizes between 1,000 and 8,000 events, and compare insertion-sort comparisons on nearly sorted and random alerts. Explain whether the measurements agree with your analysis.

(e) Deliver Production-Quality Code (8 Marks): Submit the final, optimised module with docstrings, meaningful comments and a test harness that checks every function, including the boundary cases from (b). Commit the code to your GitHub repository and share the link through Google Classroom.

Rubrics – Debugging and Optimisation Task

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Code of Conduct

I aluri poojitha(192511230)_____ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: poojitha_____ Date: 29-10-26_____

(a) Reproduce and Identify the Defects (10 marks)

Test data

```
alerts    = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries   = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log, pattern = "FAILFAIXFAIL", "FAIL"
events    = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]   # km
```

Original code (indentation restored)

```
import math


def sort_alerts(alerts):  # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j > 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j] = key
    return alerts


def is_blocked(blocklist, ip):  # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return False
        else:
            return False


def find_signature(log, pattern):  # brute-force string matching, all occurrences
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        if j == m - 1:
            hits.append(i)
```

```
    return hits


def closest_logins(events):  # brute-force closest pair of login locations
    min_dist = 0
    pair = None
    for i in range(len(events)):
        for j in range(len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d > min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist
```

Expected vs actual output

Function	Expected output	Actual output (original code)	Symptom
sort_alerts	[(A2,9), (A4,9), (A3,5), (A1,3), (A5,1)]	[(A4,9), (A3,5), (A3,5), (A5,1), (A5,1)]	A1 and A2 disappeared; A3 and A5 duplicated; A4 now before A2
is_blocked (3 queries)	[True, True, False]	[False, False, False]	Blocked IPs get through
find_signature	[0, 8]	[0, 4]	Real hit at 8 missed; false hit at 4 ("FAIX")

Defect list (nine defects)

#	Function	Faulty line	Nature of error
1	sort_alerts	while j > 0 and alerts[j][1] < key[1]:	Boundary error: index 0 is never compared or shifted. Should be j >= 0.
2	sort_alerts	alerts[j] = key	Off-by-one: the key must go at j + 1. Writing at j overwrites an element that was not moved.
3	is_blocked	if blocklist[i] == ip: return False	Wrong return value: a match must return True.

#	Function	Faulty line	Nature of error
4	is_blocked	else: return False (inside the loop)	Misplaced return: the function exits after the first entry. “Not found” may only be returned after the loop ends.
5	find_signature	for i in range(n - m):	Off-by-one: the last alignment i = n - m is never tried, so a pattern at the end of the log is missed.
6	find_signature	if j == m - 1: hits.append(i) (after the inner loop)	Logic error: when the mismatch happens on the last character, break leaves j == m - 1, so a failed alignment is recorded as a hit.
7	closest_logins	min_dist = 0	Wrong initial value: the minimum must start at infinity.
8	closest_logins	if d > min_dist:	Wrong comparison operator: finds the maximum. Should be d < min_dist.
9	closest_logins	for j in range(len(events)):	Wrong loop bounds: includes self pairs (i, i) with d = 0 and every pair twice. Should be range(i + 1, n).

(b) Root Causes and Systematic Debugging (12 marks)

Defects interact, so a partial fix can still give a wrong (or different wrong) answer. That is why each function was fixed one defect at a time and re-tested after every change.

Root causes and verification

#	Root cause and how it produces the wrong output	How it was verified
1	The while-loop stops when j reaches 0, so alerts[0] is never compared with the key. An alert that belongs in front of index 0 can never get there.	Insertion trace: with only this fix, output was still wrong (see stage table), so defect 2 was also present.

#	Root cause and how it produces the wrong output	How it was verified
2	After the loop ends, j is the index of the last element that is NOT smaller than the key (or −1). The key belongs at j + 1. Writing at j destroys that element: this is how A1 and A2 vanished and A3 and A5 were duplicated.	Trace table below: the buggy trace shows A2, A2, … after i = 1 (A1 overwritten).
3	The function answers “not blocked” exactly when it finds the IP. The security decision is inverted.	With only this fix: [False, True, False]. The first entry is now reported but the second is not (defect 4).
4	The else branch executes for the first non-matching entry, so only blocklist[0] is ever examined. 10.0.0.9 is the second entry and is never reached.	After the fix: [True, True, False]; also tested with the last entry (172.16.4.2) and an empty list.
5	For n = 12, m = 4 there are 9 alignments (0…8), but range(8) gives 0…7. The FAIL at position 8 (end of log) is never examined.	With only this fix: [0, 4, 8]: the end hit appears, but the false hit at 4 remains.
6	At alignment 4 the characters F, A, I match and X ≠ L breaks at j = 3 = m − 1. The check j == m − 1 is then true although the pattern did not match. The loop index cannot distinguish “broke at the last character” from “completed”. Fix: Python’s for … else, whose else runs only when the loop was not broken.	Alignment table below. After the fix: [0, 8].
7	Distances are never negative, so with the minimum starting at 0 no distance is ever < 0. (With the original > it instead starts the search for the maximum from 0.)	With only this fix: (None, inf), because d > inf is never true.
8	> keeps the largest distance. The report flagged logins hundreds of kilometres apart because it returned the farthest pair.	With defects 7 + 8 fixed: ((0, 0), 0.0). The self pair now wins, exposing defect 9.

#	Root cause and how it produces the wrong output	How it was verified
9	The inner loop starts at 0, so the pair (i, i) has distance 0 and is always the minimum, and every real pair is examined twice, which doubles the work.	After the fix: ((0, 5), 1.4142…), and n(n−1)/2 pairs examined.

Re-test after every correction

Function	Original	After fix 1	After fix 2	After fix 3
sort_alerts (defects 1, 2)	[[A4, 9], [A3, 5], [A3, 5], [A5, 1], [A5, 1]]	[[A1, 3], [A1, 3], [A1, 3], [A1, 3], [A4, 9]]	[[A2, 9], [A4, 9], [A3, 5], [A1, 3], [A5, 1]] ✔	–
is_blocked (3, 4)	[false, false, false]	[false, true, false]	[true, true, false] ✔	–
find_signature (5, 6)	[0, 4]	[0, 4, 8]	[0, 8] ✔	–
closest_logins (7, 8, 9)	((0, 2), 60.44)	(None, inf)	((0, 0), 0.0)	((0, 5), 1.4142) ✔

Insertion trace – original code (defects 1 and 2)

Step	Key and where the inner loop stopped	List after the step
start	-	A13, A29, A35, A49, A51
i=1	key=A2(9), stop j=0	A2(9), A2(9), A3(5), A4(9), A5(1)
i=2	key=A3(5), stop j=1	A2(9), A3(5), A3(5), A4(9), A5(1)
i=3	key=A4(9), stop j=0	A4(9), A3(5), A3(5), A3(5), A5(1)
i=4	key=A5(1), stop j=3	A4(9), A3(5), A3(5), A5(1), A5(1)

At i = 1 the loop stops at j = 0 (because of j > 0) and alerts[0] = key overwrites A1 with A2. The lost alert is never recovered.

Insertion trace – corrected code

Step	Key and where the inner loop stopped	List after the step

Step	Key and where the inner loop stopped	List after the step
start	-	A13, A29, A35, A49, A51
i=1	key=A2(9), stop j=-1	A2(9), A1(3), A3(5), A4(9), A5(1)
i=2	key=A3(5), stop j=0	A2(9), A3(5), A1(3), A4(9), A5(1)
i=3	key=A4(9), stop j=0	A2(9), A4(9), A3(5), A1(3), A5(1)
i=4	key=A5(1), stop j=3	A2(9), A4(9), A3(5), A1(3), A5(1)

At i = 3 the key A4 (9) stops at j = 0 because 9 < 9 is false. It is placed after A2, so equal severities keep arrival order (stability).

Alignment table for find_signature (log = FAILFAIXFAIL, pattern = FAIL, n = 12, m = 4)

i	Window	Comparisons	Result	Original code reports?	Corrected
0	FAIL	F=F A=A I=I L=L	MATCH	yes	hit
1	AILF	A≠F	mismatch at j=0	no	-
2	ILFA	I≠F	mismatch at j=0	no	-
3	LFAI	L≠F	mismatch at j=0	no	-
4	FAIX	F=F A=A I=I X≠L	mismatch at j=3	yes	-
5	AIXF	A≠F	mismatch at j=0	no	-
6	IXFA	I≠F	mismatch at j=0	no	-
7	XFAI	X≠F	mismatch at j=0	no	-
8	FAIL	F=F A=A I=I L=L	MATCH	not scanned	hit
9	AIL	-	cannot fit	-	-
10	IL	-	cannot fit	-	-
11	L	-	cannot fit	-	-

The original loop never scans i = 8 (range(n − m) stops at 7) and reports i = 4 because the mismatch occurred at j = m − 1.

Boundary tests (all pass on the corrected code)

Case	Input	Expected = actual
Empty blocklist	is_blocked([], "1.1.1.1")	False

Case	Input	Expected = actual
Last blocklist entry	"172.16.4.2" in the test blocklist	True
Pattern at the very end	find_signature("xxxxFAIL", "FAIL")	[4]
Pattern equals log	find_signature("FAIL", "FAIL")	[0]
Pattern longer than log	find_signature("FAI", "FAIL")	[] (no crash)
Empty log / empty pattern	find_signature("", "FAIL") / ("abc", "")	[] / []
Overlapping matches	find_signature("aaaa", "aa")	[0, 1, 2]
Equal severities	[("a",5), ("b",5), ("c",5)]	unchanged order (stable)
Empty / single alert list	[] / [("X",4)]	[] / [("X",4)]
New maximum arrives last	[("a",1), ("b",2), ("c",3)]	[c, b, a] (index 0 handled)
Fewer than two login events	closest_logins([]) and ([(1,1)])	(None, None)
Duplicate location	[(1,1), (9,9), (1,1)]	((0, 2), 0.0)

(c) Optimise the Corrected Module (10 marks)

Every optimisation was checked against the corrected version: the outputs are identical on the test data and on randomised tests.

1. sort_alerts: is insertion sort still suitable?

Corrected insertion sort does $n - 1 + I$ comparisons, where I is the number of inversions (out-of-order pairs). For **nearly sorted** alerts I is small, so it is effectively linear and moves very little data. It is also stable and in place. For arbitrary arrival order, however, I averages about $n^2/4$. At 50,000 alerts that is roughly 6.25×10^8 comparisons, which is too slow. **Decision:** the production function uses Python's stable Timsort (list.sort(key=…, reverse=True)): O(n) on nearly sorted data, O(n log n) in the worst case (about 7.8×10^5 comparisons for 50,000 alerts), and reverse=True still preserves arrival order for equal severities. The corrected insertion sort is kept as sort_alerts_insertion for the study in part (d).

2. is_blocked: repeated lookups

A sequential search costs up to m comparisons per query, so Q queries cost O(Q·m). The blocklist does not change between queries, so it is built **once** into a hash set (make_blocklist, O(m)); each lookup is then O(1) expected. The answers are identical to the corrected linear search.

3. find_signature: no redundant comparisons

The corrected scan tries exactly the n − m + 1 possible alignments, abandons an alignment at its first mismatch (break), and decides "match" with for … else instead of re-testing j. No character is compared after a failed alignment and no alignment that cannot fit is examined. Patterns longer than the log, or empty, return immediately. This stays a brute-force algorithm (worst case O(n·m), best case O(n)); a smarter matcher such as KMP is outside the CO2 outcome.

4. closest_logins: pairs, square roots and pruning

- **No self or repeated pairs:** the inner loop starts at i + 1, so only n(n−1)/2 pairs exist (the original examined n^2).
- **No square roots in the loop:** sqrt is monotonic, so comparing squared distances picks the same pair. One sqrt is taken at the end.
- **Pruning:** points are sorted by x. For a point, scanning stops as soon as the x-gap squared exceeds the best squared distance found so far; every later point is even farther in x, so it cannot be closer. The test is > rather than ≥ so equal-distance ties are still seen.
- **Tie-breaking:** a tie goes to the smaller (i, j), which is exactly what the corrected brute force returns, so outputs do not change.
- **Limit of the pruning:** if all x values are (nearly) equal, nothing is pruned and the cost is still $O(n^2)$. The harness includes this vertical-line case.

(d) Analyse Before and After (10 marks)

Operation counts and complexity

Function	Before (corrected baseline)	After optimisation
sort_alerts	Comparisons: between n−1 (sorted) and n(n−1)/2 (reverse order); average ≈ $n^2/4$. Element moves = I. Time: O(n) best, $O(n^2)$ worst/average.	Timsort: O(n) when nearly sorted, ≤ ≈ $n \cdot \log_2 n$ comparisons otherwise. Time: O(n log n) worst. Space O(n) worst.
is_blocked (Q queries, blocklist size m)	Up to m comparisons per query, Q·m in total. Time: O(Q·m).	m insertions once + Q lookups. Time: O(m + Q) expected.
find_signature	At most m(n−m+1) character comparisons, at least n−m+1. Time: O(n·m) worst, O(n) best.	Same bounds and same big-O, but no wasted checks. Measured: far closer to the best case on typical logs.

Function	Before (corrected baseline)	After optimisation
closest_logins (original loops)	n^2 distance computations and n^2 square roots (n self pairs, every pair twice). Time: $\Theta(n^2)$.	–
closest_logins (corrected brute force → optimised)	$n(n-1)/2$ distances, each with a sqrt. Time: $\Theta(n^2)$.	O(n log n) sort + pruned scan with no sqrt in the loop. Worst case $O(n^2)$ (all x equal); close to O(n log n) for spread-out points.

Measurement 1: closest_logins, distance computations (random points in a 1000 km × 1000 km square, seed 2026)

Events n	Original loops (n^2)	Corrected brute force $n(n-1)/2$ (measured)	Optimised (measured)	Reduction vs brute force	Time brute force	Time optimised
1,000	1,000,000	499,500	1,698	×294	0.12 s	0.8 ms
2,000	4,000,000	1,999,000	2,834	×705	0.48 s	1.5 ms
4,000	16,000,000	7,998,000	7,450	×1,074	1.96 s	3.7 ms
8,000	64,000,000	31,996,000	5,700	×5,613	9.26 s	6.5 ms

For every size, both versions returned the same pair and distance (asserted in the measurement script). The brute-force count equals $n(n-1)/2$ exactly. The optimised count is tiny and does not grow like n^2: it depends on how small the best distance becomes, which is why n = 8,000 needs fewer computations than n = 4,000. Time (including the sort) falls by roughly three orders of magnitude.

Measurement 2: insertion-sort comparisons, nearly sorted vs random alerts (severity 1–1000)

Alerts n	Random: comparisons	Random: $n(n-1)/4$	Nearly sorted: comparisons	Nearly sorted: inversions I	Nearly sorted: $n-1+I$
1,000	257,138	249,750	1,174	175	1,174
2,000	975,292	999,500	2,323	324	2,323
4,000	4,002,551	3,999,000	4,514	515	4,514

“Nearly sorted” means a list already ordered by severity with 5% of positions disturbed by swaps of elements at most 5 places apart.

Measurement 3: blocklist lookups and signature scan

Test	Before	After
50,000 queries against 1,000 blocked IPs (30% hits)	42,415,941 comparisons, 3.83 s	51,000 operations (build + lookups), 5.3 ms
find_signature on the test log (n = 12, m = 4)	18 character comparisons over 9 alignments (worst case 36)	same count: the corrected scan was already minimal for brute force
find_signature on a random 100,000-character log, m = 6	worst-case bound 599,970	114,450 measured (≈ 1.14 per character)

Do the measurements agree with the analysis?

- **closest_logins:** yes. Brute force matches $n(n-1)/2$ exactly (and is exactly half of the n^2 of the original loops, minus n self pairs). Quadrupling n from 1,000 to 4,000 multiplies brute-force work by about 16, as $\Theta(n^2)$ predicts. The pruned scan does not follow n^2 at all on spread-out data. The analysis said its worst case is $O(n^2)$ but typical cost is far lower, which the data confirms.
- **Insertion sort:** yes. Random data needs close to $n^2/4$ comparisons (within about 3% here). For nearly sorted data the count equals $n-1+I$ exactly (e.g. 999 + 175 = 1,174), so cost grows with disorder, not with n^2. This supports using insertion sort only for nearly sorted data.
- **Blocklist:** yes. About 4.2×10^7 comparisons versus about 5×10^4 operations, matching $O(Q \cdot m)$ against $O(m + Q)$. Timings are wall-clock in Python and vary by machine; the operation counts are the reliable evidence.
- **Signature scan:** the count is between the $n-m+1$ best case and the $m(n-m+1)$ worst case, close to the best case on random text, because most alignments fail at the first character.

(e) Production-Quality Code (8 marks)

Files to commit to the GitHub repository: sentinelops.py (final module), test_sentinelops.py (harness), and this report.

sentinelops.py

```
"""SentinelOps SOC alert-triage module (CSA06 - CO2, corrected and optimised).

Functions
---------
sort_alerts        : order alerts by severity, highest first, stable.
make_blocklist     : build a hash index of the blocklist once.
is_blocked         : membership test against that index.
find_signature     : every start position of a pattern in a log line.
closest_logins     : closest pair of login locations (index pair, distance).
```

```
"""
import math


def sort_alerts(alerts):
    """Sort (id, severity) tuples by severity, highest first.

    Stable: alerts of equal severity keep their arrival order.
    Sorts in place and returns the same list, as the original did.
    Timsort is O(n) on nearly sorted input and O(n log n) in the worst case;
    reverse=True keeps equal keys in their original order.
    """
    alerts.sort(key=lambda alert: alert[1], reverse=True)
    return alerts


def sort_alerts_insertion(alerts):
    """Corrected insertion sort (kept for the comparison study in part (d)).

    Strict '<' means an equal-severity alert is never moved past another,
    which is what makes the sort stable.
    """
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:   # j >= 0 so index 0 is shifted too
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key                       # slot just after the last shift
    return alerts


def make_blocklist(blocklist):
    """Build the hash index once: O(m). Reuse it for every lookup."""
    return frozenset(blocklist)


def is_blocked(index, ip):
```

```
    """Return True if ip is on the blocklist (index from make_blocklist). O(1) expected."""
    return ip in index


def is_blocked_linear(blocklist, ip):
    """Corrected sequential search (kept for the comparison study). O(m)."""
    for entry in blocklist:
        if entry == ip:
            return True      # found: report BLOCKED
    return False             # only after every entry has been checked


def find_signature(log, pattern):
    """Return every start index where pattern occurs in log (overlaps allowed).

    Alignments 0 .. n-m inclusive are tried; the inner loop stops at the first
    mismatch, so no character is compared after an alignment has failed.
    An empty pattern or a pattern longer than the log gives [].
    """
    n, m = len(log), len(pattern)
    if m == 0 or m > n:
        return []
    hits = []
    for i in range(n - m + 1):           # +1: last alignment starts at n-m
        for j in range(m):
            if log[i + j] != pattern[j]:
                break                    # mismatch: abandon this alignment
        else:                            # runs only if the loop was never broken
            hits.append(i)
    return hits


def closest_logins(events):
    """Return ((i, j), distance) for the closest pair of (x, y) points, i < j.

    Points are sorted by x. For each point only later points in that order are
    examined, and the scan stops once the x-gap alone exceeds the best distance.
    Squared distances are compared; one sqrt is taken at the end.
```

```
    Ties are resolved to the lexicographically smallest (i, j), so the result
    equals the corrected brute-force version. Fewer than two events -> (None, None).
    """
    n = len(events)
    if n < 2:
        return None, None
    order = sorted(range(n), key=lambda k: events[k][0])
    best_sq = math.inf
    best_pair = None
    for a in range(n):
        ia = order[a]
        xa, ya = events[ia]
        for b in range(a + 1, n):
            ib = order[b]
            dx = events[ib][0] - xa
            if dx * dx > best_sq:          # all later points are even farther in x
                break
            dy = events[ib][1] - ya
            d_sq = dx * dx + dy * dy
            pair = (ia, ib) if ia < ib else (ib, ia)
            if d_sq < best_sq or (d_sq == best_sq and pair < best_pair):
                best_sq, best_pair = d_sq, pair
    return best_pair, math.sqrt(best_sq)


def closest_logins_bruteforce(events):
    """Corrected brute-force version (reference for testing and part (d))."""
    n = len(events)
    if n < 2:
        return None, None
    min_dist = math.inf
    pair = None
    for i in range(n):
        for j in range(i + 1, n):          # no self pairs, no repeated pairs
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d < min_dist:               # closest, so '<' not '>'
                min_dist = d
```

```
                pair = (i, j)
    return pair, min_dist
```

test_sentinelops.py

```
"""Test harness for sentinelops.py - run with:  python test_sentinelops.py"""
import math
import random
import sentinelops as s

passed = failed = 0

def check(name, actual, expected):
    """Record one test result and print PASS/FAIL."""
    global passed, failed
    ok = actual == expected
    passed += ok
    failed += not ok
    print(("PASS" if ok else "FAIL"), "-", name, "" if ok else f"\n   expected {expected!r}\n   actual
{actual!r}")

ALERTS = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
SORTED = [("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)]

# ---- sort_alerts (fast) and the corrected insertion sort ----
for fn in (s.sort_alerts, s.sort_alerts_insertion):
    check(f"{fn.__name__}: test data", fn(list(ALERTS)), SORTED)
    check(f"{fn.__name__}: empty list", fn([]), [])
    check(f"{fn.__name__}: single alert", fn([("X", 4)]), [("X", 4)])
    check(f"{fn.__name__}: all equal severity keeps arrival order",
          fn([("a", 5), ("b", 5), ("c", 5)]), [("a", 5), ("b", 5), ("c", 5)])
    check(f"{fn.__name__}: new maximum at the end moves to index 0",
          fn([("a", 1), ("b", 2), ("c", 3)]), [("c", 3), ("b", 2), ("a", 1)])
    check(f"{fn.__name__}: already sorted", fn(list(SORTED)), SORTED)
random.seed(1)
big = [(f"A{i}", random.randint(1, 10)) for i in range(2000)]
check("both sorts agree on 2000 random alerts",
      s.sort_alerts(list(big)), s.sort_alerts_insertion(list(big)))
check("sort result is stable on 2000 random alerts",
```

```
    s.sort_alerts(list(big)), sorted(big, key=lambda a: -a[1]))

# ---- is_blocked ----
BL = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
idx = s.make_blocklist(BL)
for q, exp in [("10.0.0.9", True), ("10.0.0.5", True), ("8.8.8.8", False),
               ("172.16.4.2", True)]:          # last entry must be found too
    check(f"is_blocked({q})", s.is_blocked(idx, q), exp)
    check(f"is_blocked_linear({q})", s.is_blocked_linear(BL, q), exp)
check("empty blocklist (hash)", s.is_blocked(s.make_blocklist([]), "1.1.1.1"), False)
check("empty blocklist (linear)", s.is_blocked_linear([], "1.1.1.1"), False)
check("single-entry blocklist hit", s.is_blocked(s.make_blocklist(["a"]), "a"), True)

# ---- find_signature ----
check("test data", s.find_signature("FAILFAIXFAIL", "FAIL"), [0, 8])
check("pattern at the very end", s.find_signature("xxxxFAIL", "FAIL"), [4])
check("pattern is the whole log", s.find_signature("FAIL", "FAIL"), [0])
check("pattern longer than log", s.find_signature("FAI", "FAIL"), [])
check("empty log", s.find_signature("", "FAIL"), [])
check("empty pattern", s.find_signature("abc", ""), [])
check("overlapping matches", s.find_signature("aaaa", "aa"), [0, 1, 2])
check("no match, last char differs", s.find_signature("FAIX", "FAIL"), [])
check("single-character pattern", s.find_signature("abcab", "b"), [1, 4])

# ---- closest_logins ----
EV = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
for fn in (s.closest_logins, s.closest_logins_bruteforce):
    pair, d = fn(EV)
    check(f"{fn.__name__}: test data pair", pair, (0, 5))
    check(f"{fn.__name__}: test data distance", round(d, 6), round(math.sqrt(2), 6))
    check(f"{fn.__name__}: no events", fn([]), (None, None))
    check(f"{fn.__name__}: one event", fn([(1, 1)]), (None, None))
    check(f"{fn.__name__}: two events", fn([(0, 0), (3, 4)]), ((0, 1), 5.0))
    check(f"{fn.__name__}: duplicate location gives distance 0", fn([(1, 1), (9, 9), (1, 1)]), ((0, 2),
0.0))
    check(f"{fn.__name__}: all x equal (vertical line)", fn([(5, 0), (5, 10), (5, 11), (5, 30)]), ((1, 2),
1.0))
random.seed(7)
```

```
ok = True
for _ in range(300):
    pts = [(random.randint(0, 30), random.randint(0, 30)) for _ in range(random.randint(2, 40))]
    ok &= s.closest_logins(pts) == s.closest_logins_bruteforce(pts)   # incl. tie-breaking
check("optimised == brute force on 300 random inputs with many ties", ok, True)


print(f"\n{passed} passed, {failed} failed")
raise SystemExit(1 if failed else 0)
```

Harness output

```
PASS - sort_alerts: test data
PASS - sort_alerts: empty list
PASS - sort_alerts: single alert
PASS - sort_alerts: all equal severity keeps arrival order
PASS - sort_alerts: new maximum at the end moves to index 0
PASS - sort_alerts: already sorted
PASS - sort_alerts_insertion: test data
PASS - sort_alerts_insertion: empty list
PASS - sort_alerts_insertion: single alert
PASS - sort_alerts_insertion: all equal severity keeps arrival order
PASS - sort_alerts_insertion: new maximum at the end moves to index 0
PASS - sort_alerts_insertion: already sorted
PASS - both sorts agree on 2000 random alerts
PASS - sort result is stable on 2000 random alerts
PASS - is_blocked(10.0.0.9)
PASS - is_blocked_linear(10.0.0.9)
PASS - is_blocked(10.0.0.5)
PASS - is_blocked_linear(10.0.0.5)
PASS - is_blocked(8.8.8.8)
PASS - is_blocked_linear(8.8.8.8)
PASS - is_blocked(172.16.4.2)
PASS - is_blocked_linear(172.16.4.2)
PASS - empty blocklist (hash)
PASS - empty blocklist (linear)
PASS - single-entry blocklist hit
PASS - test data
PASS - pattern at the very end
PASS - pattern is the whole log
```

```
PASS - pattern longer than log
PASS - empty log
PASS - empty pattern
PASS - overlapping matches
PASS - no match, last char differs
PASS - single-character pattern
PASS - closest_logins: test data pair
PASS - closest_logins: test data distance
PASS - closest_logins: no events
PASS - closest_logins: one event
PASS - closest_logins: two events
PASS - closest_logins: duplicate location gives distance 0
PASS - closest_logins: all x equal (vertical line)
PASS - closest_logins_bruteforce: test data pair
PASS - closest_logins_bruteforce: test data distance
PASS - closest_logins_bruteforce: no events
PASS - closest_logins_bruteforce: one event
PASS - closest_logins_bruteforce: two events
PASS - closest_logins_bruteforce: duplicate location gives distance 0
PASS - closest_logins_bruteforce: all x equal (vertical line)
PASS - optimised == brute force on 300 random inputs with many ties

49 passed, 0 failed
```

Appendix – Program Runs (console)

Both programs are single self-contained files (module code plus a driver), so they can be pasted into any online Python compiler and run as they are.

Run 1: original (buggy) module – driver code

```
# ---- Driver: run the ORIGINAL (buggy) module on the test data ----
alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log, pattern = "FAILFAIXFAIL", "FAIL"
events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]

print("sort_alerts    :", sort_alerts(list(alerts)))
print("is_blocked     :", [is_blocked(blocklist, q) for q in queries])
```

```
print("find_signature :", find_signature(log, pattern))
print("closest_logins :", closest_logins(events))
```

Output:

```
sort_alerts    : [('A4', 9), ('A3', 5), ('A3', 5), ('A5', 1), ('A5', 1)]
is_blocked     : [False, False, False]
find_signature : [0, 4]
closest_logins : ((0, 2), 60.440052945046304)
```

Run 2: corrected and optimised module – driver code

The module code from sentinelops.py (above) is placed first in the same file; the driver below follows it.

```
# ---- Driver: run the CORRECTED + OPTIMISED module on the test data ----
import random
alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log, pattern = "FAILFAIXFAIL", "FAIL"
events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]

print("sort_alerts          :", sort_alerts(list(alerts)))
print("sort_alerts_insertion :", sort_alerts_insertion(list(alerts)))
idx = make_blocklist(blocklist)
print("is_blocked           :", [is_blocked(idx, q) for q in queries])
print("find_signature       :", find_signature(log, pattern))
print("closest_logins       :", closest_logins(events))
print("closest (brute force) :", closest_logins_bruteforce(events))

print("\n--- boundary cases ---")
print("empty blocklist      :", is_blocked(make_blocklist([]), "1.1.1.1"))
print("pattern at end       :", find_signature("xxxxFAIL", "FAIL"))
print("pattern > log        :", find_signature("FAI", "FAIL"))
print("equal severities     :", sort_alerts([("a", 5), ("b", 5), ("c", 5)]))
print("0 / 1 login events   :", closest_logins([]), closest_logins([(1, 1)]))

print("\n--- optimised vs brute force on 2000 random events ---")
random.seed(1)
ev = [(random.uniform(0, 1000), random.uniform(0, 1000)) for _ in range(2000)]
print("same result          :", closest_logins(ev) == closest_logins_bruteforce(ev))
```

Output:

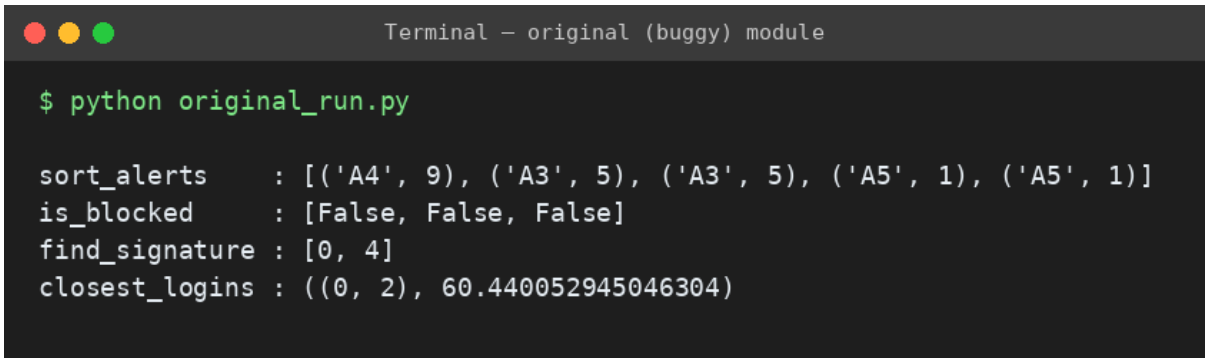
```
sort_alerts          : [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]
sort_alerts_insertion : [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]
is_blocked           : [True, True, False]
find_signature       : [0, 8]
closest_logins       : ((0, 5), 1.4142135623730951)
closest (brute force) : ((0, 5), 1.4142135623730951)

--- boundary cases ---
empty blocklist      : False
pattern at end       : [4]
pattern > log        : []
equal severities     : [('a', 5), ('b', 5), ('c', 5)]
0 / 1 login events   : (None, None) (None, None)

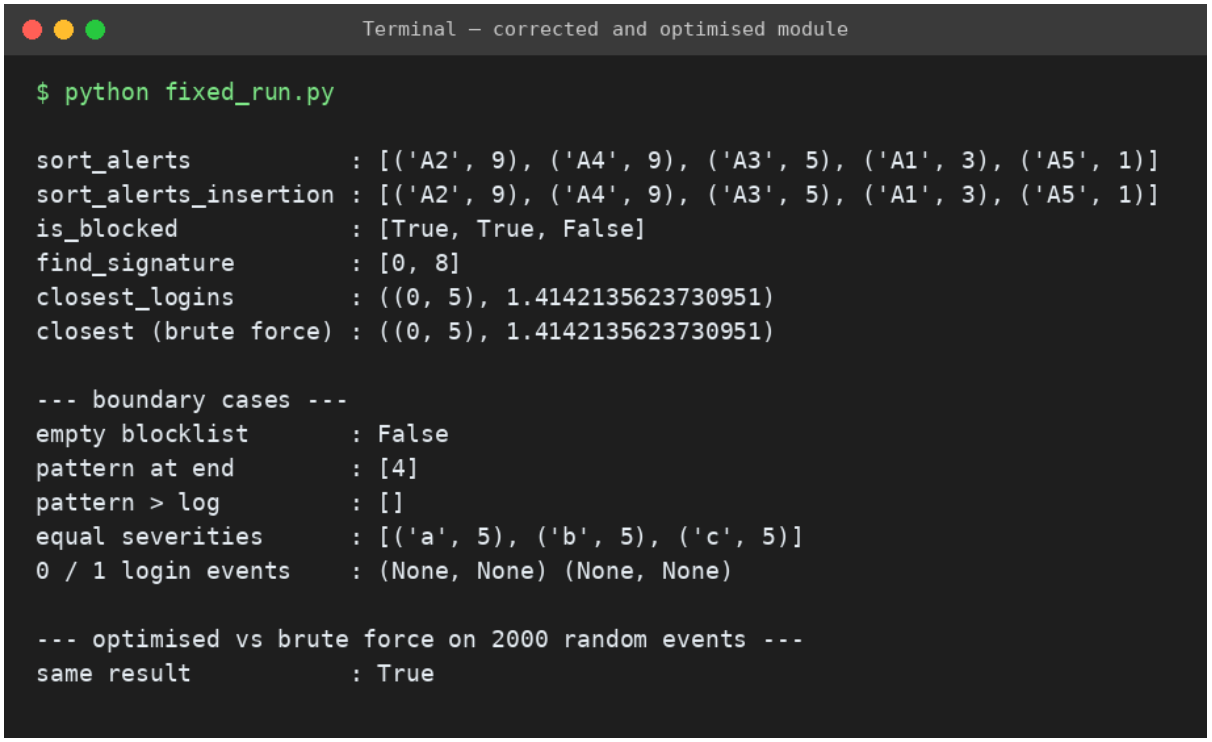
--- optimised vs brute force on 2000 random events ---
same result          : True
```

Appendix – Console Screenshots

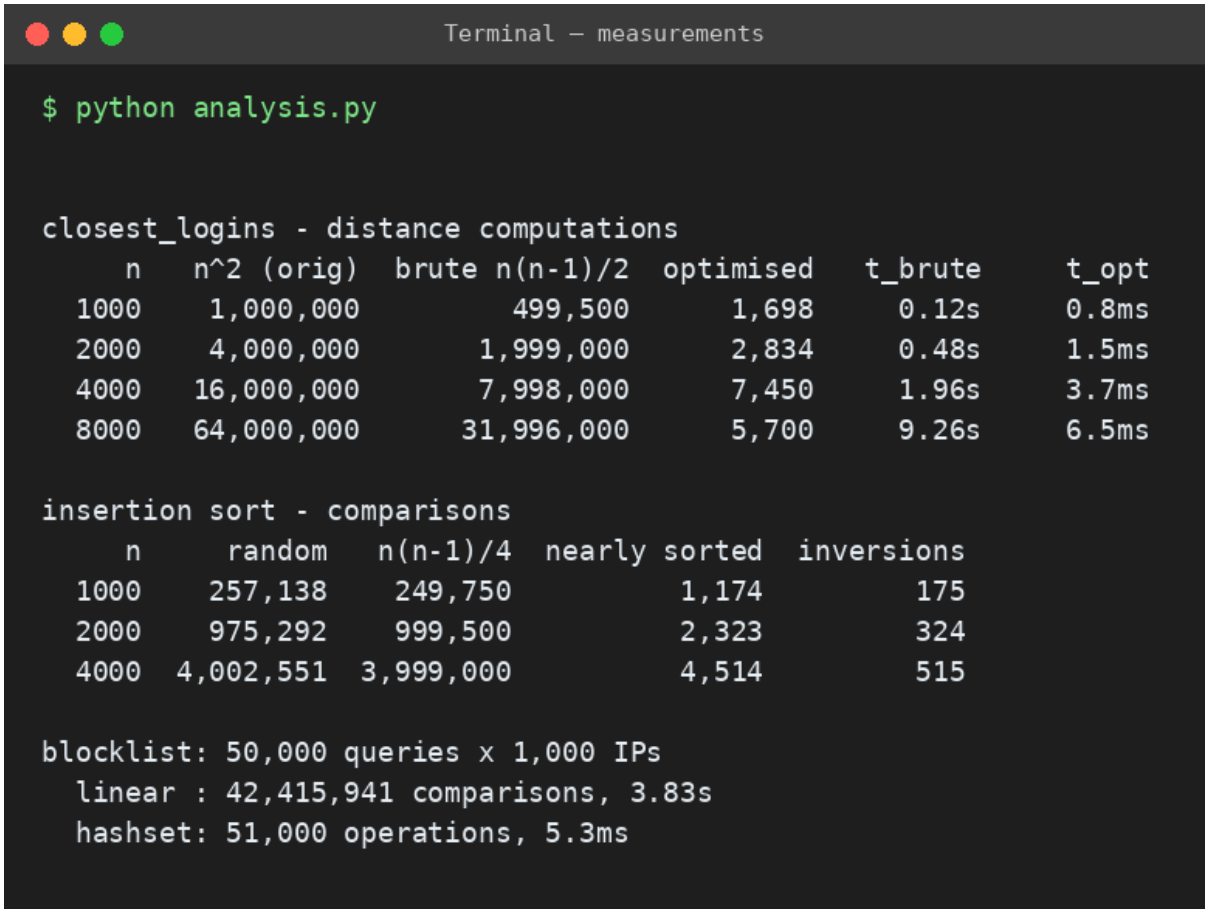
Screenshots of the console output of each run (captured from the programs listed above).



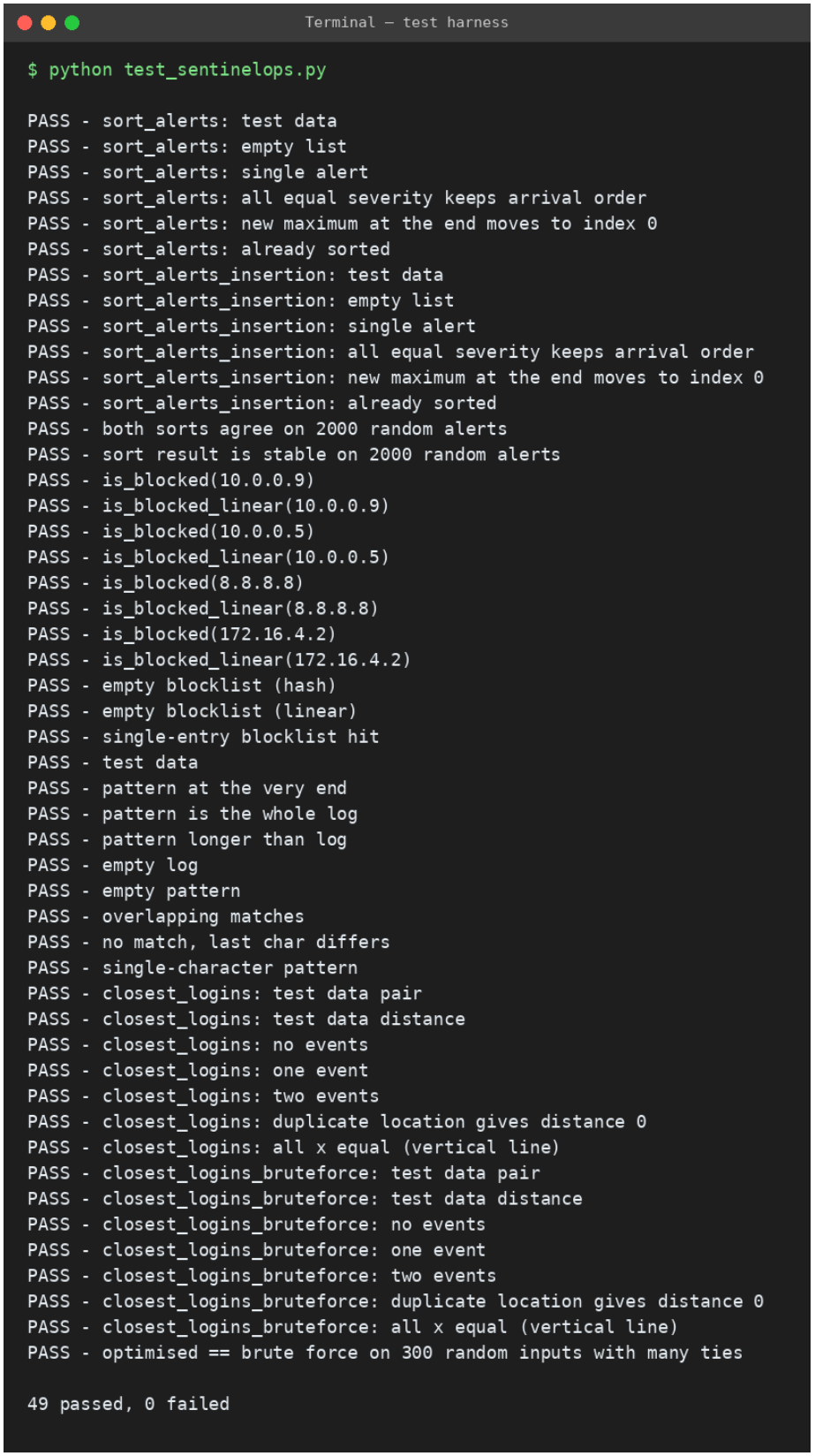
Screenshot 1: original (buggy) module on the test data



Screenshot 2: corrected and optimised module, boundary cases and brute-force check



Screenshot 3: measurements for part (d)



Screenshot 4: test harness (49 passed, 0 failed)